

DOI:10.13232/j.cnki.jnju.2017.06.016

HIGHT 轻量级分组密码容错代数故障攻击研究

陈 浩^{1*}, 王 韬¹, 赵新杰^{2,3}, 张 帆⁴, 马云飞¹, 王晓晗¹

(1.解放军军械工程学院信息工程系, 石家庄, 050003; 2.解放军外国语学院指挥系, 洛阳, 471003;
3.北方电子设备研究所, 北京, 100191; 4.浙江大学信息与电子工程学院, 杭州, 310027)

摘 要:针对现有 HIGHT 轻量级分组密码代数故障攻击方法在故障失效发生的场景下故障位置判定出错进而影响攻击成功率的问题, 提出并讨论了一种容错代数故障攻击方法. 该方法首先对故障失效特性进行深入研究, 利用故障注入位置、故障失效与密文差分之间的对应关系, 构建了一个完备的故障位置区分器以实现各种场景下故障位置的准确判定. 然后在此基础上提取故障失效信息以实现故障信息最大化利用, 并对故障信息等代数方程组构建方法进行优化, 实现了故障信息等代数方程组的自动化构建. 最后对提出的攻击方法的复杂度和成功率进行了分析和实验验证. 实验结果表明, 与现有攻击相比, 提出的攻击方法容错能力更强, 能检测出所有显性故障失效, 攻击的成功率达到 100%, 且故障信息等代数方程组构建自动化程度更高, 解析器平均求解时间更少.

关键词:轻量级分组密码, 故障相遇, 容错策略, 代数故障攻击, HIGHT

中图分类号: TP918

文献标识码: A

Research on fault-tolerant algebraic fault attack on HIGHT

Chen Hao^{1*}, Wang Tao¹, Zhao Xinjie^{2,3}, Zhang Fan⁴, Ma Yunfei¹, Wang Xiaohan¹

(1. Department of Information Engineering, Ordnance Engineering College, Shijiazhuang, 050003, China;
2. Department of Command, PLA University of Foreign Language, Luoyang, 471003, China;
3. Institute of North Electronic Equipment, Beijing, 100191, China; 4. College of Information Science
and Electronic Engineering, Zhejiang University, Hangzhou, 310027, China)

Abstract: HIGHT is built by using ARX (addition modulo 2^n , bit rotation and XOR) structure, which is suitable for resource-constrained environment such as Radio Frequency Identification (RFID) tag or ubiquitous computing system and it has been adopted as a standard block cipher by Telecommunications Technology Association (TTA) of Korea and ISO/IEC 18033-3. Since the accurate location of the injected fault cannot be successfully determined when fault failures are occurred, the success rate of the existing algebraic fault attack on HIGHT is always less than 100%. To improve the success rate and efficiency, a fault tolerant algebraic fault attack is proposed in this paper. Firstly, fault failures and its properties are studied and a complete distinguisher based on fault failures, fault locations and cipher differences for determining the accurate fault locations in all different scenarios is built. Then, HIGHT is described as a set of algebraic equations. The faulty ciphertext is generated via fault injections and fault differences are represented

基金项目:国家自然科学基金 (61173191, 61272491, 61309021, 61472357, 61571063)

收稿日期: 2017-05-02

* 通讯联系人, E-mail: chen hao81823264@163.com

with algebraic equations. To make maximum use of the injected faults, fault failures are also described as a set of algebraic equations. In the meantime, the procedure of constructing algebraic equations for the injected faults is optimized to perform automatically to further make the attack easy to launch. Finally, the CryptoMiniSAT solver is applied to solve the equations for the key and the number of fault injections that required and success rate of the proposed attack are analyzed in theory. The simulation experiments show that compared with the existing algebraic fault attack on HIGHT, the success rate of the proposed attack has been improved to 100% and the method of constructing algebraic equations for the injected faults is easier and can be performed automatically, the entire mater key bytes can be fully recovered in a rather smaller time by solving the algebraic equations with the CryptoMiniSAT solver, and the proposed attack can be easily extended to other cipher which has the similar structure.

Key words: lightweight block cipher, fault collision, fault-tolerance strategy, algebraic fault attack, HIGHT

微电子技术和普适计算的高速发展使物联网和传感器网络等逐步深入到人们生活的方方面面,如何在此类资源受限环境下提供信息安全保障成为亟待解决的重点和难点问题.在这种情况下,轻量级分组密码受到广泛关注,目前研究者已陆续提出了一些轻量级分组密码算法,包括 PRESENT^[1], HIGHT^[2], LED^[3], Piccolo^[4]等.与传统分组密码算法相比,轻量级分组密码能够以非常小的代价及成本在硬件上高效实现,同时保证一定程度的安全性,因此适用于资源受限环境.然而,由于受轻量级设计和实现原则所限,轻量级密码存在防护程度不足的不足,导致其容易遭受故障注入的威胁;此外轻量级密码算法的代数结构相对传统密码算法而言比较简单,导致其代数方程组求解复杂度相对较低,二者均为基于代数攻击和故障攻击的结合体——代数故障攻击提供了故障注入和分析的有利条件.

代数故障攻击^[5]将代数攻击和故障攻击结合起来,首先利用代数分析方法构建密码算法等效代数方程组,然后利用故障注入手段获取错误密文并将故障差分转化为代数方程组,最后通过解析器求解代数方程组恢复密钥信息,从而将密码分析过程转化为代数方程组的自动构建和求解问题.代数故障攻击一经提出便受到广泛关注,其攻击对象逐渐趋向多样化,已成功扩展至 LED^[6], PRESENT^[7], Piccolo^[8]等分组密码和 Trivium^[9]等流密码.同传统差分故

障攻击相比,代数故障攻击能够充分利用故障攻击中泄露出来的故障信息,具有攻击通用性强、方程可自动化求解的优点,为密码分析提供了一种新的自动化、通用化分析手段.

HIGHT 算法是一种典型的轻量级分组密码,设计时仅采用 ARX 结构(由模 2^n 加运算、循环移位、异或运算三种运算按照一定顺序组合而成的安全结构),执行效率高,非常适合在微型设备中使用,其安全性研究具有重要意义,自 2006 年提出以来密码界对其安全性开展了一系列研究.范伟杰等^[10]采用面向字节的故障模型,对 HIGHT 提出一种差分故障攻击方法,通过穷举 4 字节白化密钥,并分别在密码倒数第 3 轮和倒数第 4 轮共注入约 32 个故障可恢复 HIGHT 全部 128 bit 密钥信息.但由于范伟杰等^[10]采用的是传统故障分析方法,分析过程繁琐,故障信息利用率低,所需故障样本量较大,导致攻击方法有效性和通用性不强.陈浩等^[11]对 HIGHT 密码提出一种代数故障分析方法,将范伟杰等^[10]的手工分析方法转化为代数方程组自动构建和求解问题,成功将攻击轮数扩展至加密倒数第 6 轮,故障注入个数减小为 9 次.由于陈浩等^[11]利用故障密文的数量和索引来确定故障的具体位置,当故障在传播过程中发生碰撞时,陈浩等^[11]确定故障具体位置的方法有可能失效,导致攻击的成功率不能达到 100%.

针对现有攻击方法的不足,本文对 HIGHT

密码提出一种容错代数故障分析方法,将攻击成功率提升至 100%,并从理论角度分析且给出了攻击所需故障注入次数和成功率.在密码第 26 轮加密注入单字节随机故障下,11 次故障注入以 100% 成功率恢复全部主密钥信息,平均求解时间仅为 2.18 s,最好情况下仅需 4 次即能以成功率 92%,在 124.6 s 内恢复全部主密钥信息.此外,本文还对密码 27 轮加密进行了攻击,恢复全部主密钥信息的故障注入次数为 23 次,实际成功率均为 100%,与理论分析基本一致.与现有 HIGHT 攻击相比,文中方法将攻击成功率提升至 100%,且攻击自动化程度较高.

1 HIGHT 密码算法

1.1 符号说明

$\oplus, \boxplus, \parallel$ 异或运算,模 2^8 加运算和变量连接符;

$A \lll \alpha$: 循环左移 α 位;

$'$: 注入故障之后密码的内部状态或输出;

$\#S$: 有限集合 S 的势;

P, C : 64 比特明文和密文;

ΔC_i : 正误密文差分值, $\Delta C_i = C_i \oplus C'_i, 0 \leq i \leq 7$;

$MK = MK_{15} \parallel \cdots \parallel MK_1 \parallel MK_0$: 16 字节主密钥;

WK_i : 白化密钥 $WK_i, 0 \leq i \leq 7$;

SK_k : 轮子密钥, $0 \leq k \leq 127$;

$X_r = X_{r,7} \parallel \cdots \parallel X_{r,1} \parallel X_{r,0}$: 第 r 轮 8 字节中间状态, $1 \leq r \leq 32$;

$X_{r,i}^k$: 变量 $X_{r,i}^k$ 第 k 比特, $0 \leq k \leq 7$;

f_d : 故障深度, 表示故障注入轮 r 到错误密文之间的传播轮数, $f_d = 32 - r + 1, 1 \leq f_d \leq 32$.

1.2 HIGHT 加密过程 HIGHT 是一个 32 轮迭代轻量级分组密码算法,其分组长度和密钥长度分别为 64 和 128 位. HIGHT 密码由密钥扩展、初始变换、轮加密、末轮变换 4 个子过程构成:

(1) 密钥扩展. 根据主密钥 MK 按照式 (1)~(4) 生成 8 字节白化密钥 WK_i 和每轮加

密使用的 4 字节子密钥 SK_i :

$$WK_i = MK_{i+12} (i=0,1,2,3) \quad (1)$$

$$WK_i = MK_{i-4} (i=4,5,6,7) \quad (2)$$

$$SK_{16 \cdot i + j} = MK_{j-i \bmod 8} \boxplus \delta_{16 \cdot i + j} \quad (0 \leq i, j \leq 7) \quad (3)$$

$$SK_{16 \cdot i + j + 8} = MK_{(j-i \bmod 8) + 8} \boxplus \delta_{16 \cdot i + j + 8} \quad (0 \leq i, j \leq 7) \quad (4)$$

(2) 初始变换. 明文 P 根据式 (5)~(6), 经初始变换成加密首轮输入 X_0 :

$$X_{0,0} = P_0 \boxplus WK_0; X_{0,1} = P_1;$$

$$X_{0,2} = P_2 \oplus WK_1; X_{0,3} = P_3 \quad (5)$$

$$X_{0,4} = P_4 \boxplus WK_2; X_{0,5} = P_5;$$

$$X_{0,6} = P_6 \oplus WK_3; X_{0,7} = P_7 \quad (6)$$

(3) 轮加密函数. 执行 32 轮加密, 将第 i ($0 \leq i \leq 30$) 轮输入 X_i 按照式 (7)~(14) 转换成第 $i+1$ 轮输入 X_{i+1} :

$$X_{i+1,0} = X_{i,7} \oplus (F_0(X_{i,6}) \boxplus SK_{4i+3}) \quad (7)$$

$$X_{i+1,1} = X_{i,0} \quad (8)$$

$$X_{i+1,2} = X_{i,1} \boxplus (F_1(X_{i,0}) \oplus SK_{4i}) \quad (9)$$

$$X_{i+1,3} = X_{i,2} \quad (10)$$

$$X_{i+1,4} = X_{i,3} \oplus (F_0(X_{i,2}) \boxplus SK_{4i+1}) \quad (11)$$

$$X_{i+1,5} = X_{i,4} \quad (12)$$

$$X_{i+1,6} = X_{i,5} \boxplus (F_0(X_{i,4}) \oplus SK_{4i+2}) \quad (13)$$

$$X_{i+1,7} = X_{i,6} \quad (14)$$

最后一轮 ($i=31$) 轮加密的轮函数为:

$$X_{i+1,0} = X_{i,0} \quad (15)$$

$$X_{i+1,1} = X_{i,1} \boxplus (F_1(X_{i,0}) \oplus SK_{4i}) \quad (16)$$

$$X_{i+1,2} = X_{i,2} \quad (17)$$

$$X_{i+1,3} = X_{i,3} \oplus (F_0(X_{i,2}) \boxplus SK_{4i+1}) \quad (18)$$

$$X_{i+1,4} = X_{i,4} \quad (19)$$

$$X_{i+1,5} = X_{i,5} \boxplus (F_1(X_{i,4}) \oplus SK_{4i+2}) \quad (20)$$

$$X_{i+1,6} = X_{i,6} \quad (21)$$

$$X_{i+1,7} = X_{i,7} \oplus (F_0(X_{i,6}) \boxplus SK_{4i+3}) \quad (22)$$

F_0 和 F_1 表达式如式 (23)~(24) 所示:

$$F_0(x) = x \lll 1 \oplus x \lll 2 \oplus x \lll 7 \quad (23)$$

$$F_1(x) = x \lll 3 \oplus x \lll 4 \oplus x \lll 6 \quad (24)$$

(4) 末轮变换. 算法在执行完 32 轮加密运算后, 将 X_{32} 根据式 (25) 经末变换生成密文 C :

$$C_0 = X_{32,1} \boxplus WK_4; C_1 = X_{32,2}$$

$$C_2 = X_{32,3} \oplus WK_4; C_3 = X_{32,4}$$

$$\begin{aligned} C_4 &= X_{32,5} \oplus WK_6; C_5 = X_{32,6} \\ C_6 &= X_{32,7} \oplus WK_7; C_7 = X_{32,0} \end{aligned} \quad (25)$$

2 HIGHT 代数故障分析

2.1 故障模型设定 本攻击采用面向单字节的随机故障模型,如图 1 所示,其基本假设为:

攻击者能够利用低成本故障注入技术(如时钟毛刺、亏电效应、电压峰值等^[12])向 HIGHT 加密第 r 轮注入单字节随机故障,但故障的具体值和具体位置均未知.此外,攻击者能够选择被加密的明文,并且获取同一个主密钥作用下的正确密文和错误密文.

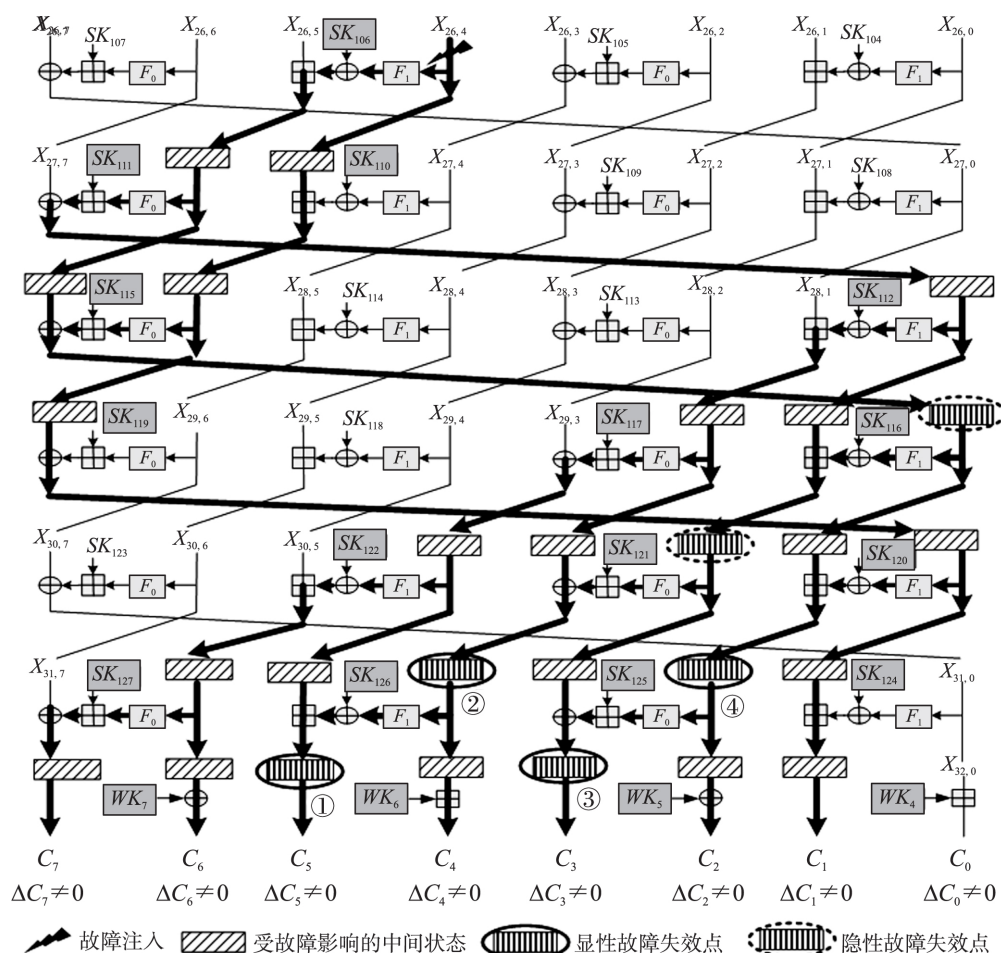


Fig.1 Fault model of the attack

由图 1 可知,当攻击者向 HIGHT 第 26 轮加密注入单字节随机故障时,故障传播特征非常复杂,很难应用传统差分故障分析方法进行密钥分析.

2.2 故障传播特性分析

性质 1 假设攻击者向密码第 r ($0 \leq r < 32$) 轮加密的 $X_{r,\alpha}$ 中间状态变量注入单字节随机故障,定义集合:

$$\Phi(f_d, \alpha)_n =$$

$$\{X_{r,i} \mid \Delta X_{r,i} \neq 0, 0 \leq i, \alpha \leq 7, r \leq n \leq 32\}$$

为故障传播过程中,第 n 轮加密中受影响的中间状态变量,则有下列结论成立,其中 $0 \leq m \leq 3$:

$$(1) \# \Phi(r, \alpha)_r = 1;$$

$$(2) \# \Phi(r, \alpha)_{n+1} =$$

$$\begin{cases} \# \Phi(r, \alpha)_n + 1, & \alpha = 2m \\ \# \Phi(r, \alpha)_n, & \alpha = 2m + 1 \end{cases}$$

$$(3) \# \Phi(r, \alpha)_{n+1} = \begin{cases} \# \Phi(r, \alpha)_n + 1, & \# \Phi(r, \alpha)_n < 7 \\ 8, & \# \Phi(r, \alpha)_n = 8 \end{cases}$$

证明 由故障模型知, 攻击者每次仅注入一个故障, 因此结论(1)显然成立.

对于结论(2):

① 当 $\alpha = 2m$ 时, 如图 2 所示, 可知下一轮故障将向 $X_{r+1, 2(m+1)}$ 和 $X_{r+1, 2m+1}$ 扩散, 所以有 $\# \Phi(r, 2m)_{r+1} = \# \Phi(r, 2m)_r + 1$.

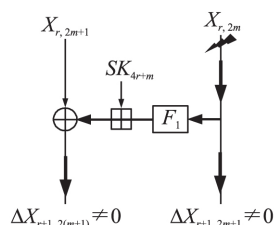


图 2 故障位置为 $X_{r, 2m}$

Fig.2 The fault is injected in $X_{r, 2m}$

② 当 $\alpha = 2m+1$ 时, 如图 3 所示, 可知下一轮故障将仅向一个中间状态变量 $X_{r+2, 2(m+1)}$ 扩散, 故有 $\# \Phi(r, 2m+1)_{r+1} = \# \Phi(r, 2m+1)_r$.

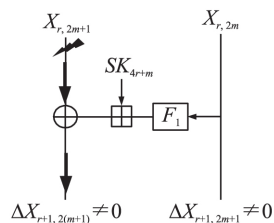


图 3 故障位置为 $X_{r, 2m+1}$

Fig.3 The fault is injected in $X_{r, 2m+1}$

对于结论(3), 由结论(2)证明过程可知, 当故障没有影响密码第 n 轮加密全部 8 字节中间状态变量时, 显然有 $\# \Phi(r, \alpha)_{n+1} = \# \Phi(r, \alpha)_n + 1, n > r$. 反之, 则有 $\# \Phi(r, \alpha)_{n+1} = \# \Phi(r, \alpha)_n = 8$, 所以结论(3)成立.

证毕.

性质 2 故障相遇 如图 4 所示, 假设攻击者向密码中间状态变量 $X_{i, 2m}$ 注入一个单字节随机故障 β . 故障在传播过程中将会影响密码中间状态变量的值, 如果单故障注入后导致的多个差分同时传播到某个中间状态变量 $X_{r, k}$ 时, 称之为故障相遇. 如果这多个故障差分同时

作用于该中间状态变量且故障相互之间发生抵消使 $\Delta X_{r, k} = 0$, 则称之为故障失效.

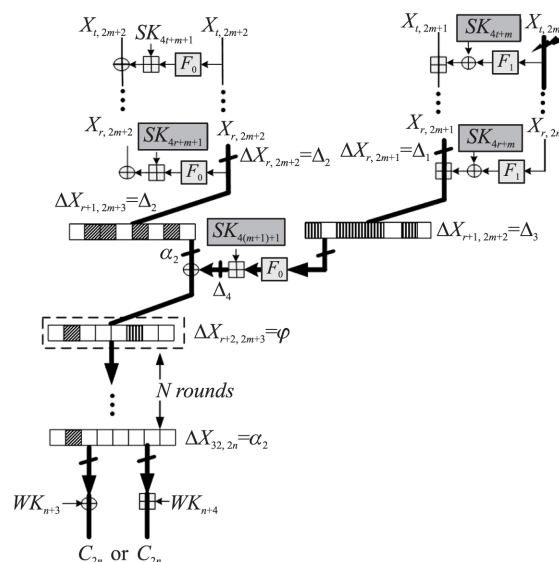


图 4 故障相遇和故障失效示意图

Fig.4 Fault collision and fault failure

例如, 在图 4 中, 故障差分 Δ_3 和 Δ_4 同时传播到 $X_{r+2, 2m+3}$ 时发生故障相遇, 当故障差分 Δ_3 和 Δ_4 相互抵消使 $\Delta_5 = 0$ 时发生故障失效, 此时故障对 $X_{r+2, 2m+3}$ 的值无影响.

性质 3 显性故障失效和隐性故障失效

如果故障失效引起密文差分值发生改变, 则称之为显性故障失效. 反之, 则称之为隐性故障失效.

例如, 在图 4 中, 假设故障失效发生在中间状态 $X_{r+2, 2m+3}$, 当 $N=1$ 时, $\Delta C_{2n} = 0$, 此时故障失效引起密文差分值发生改变, 该故障失效为显性故障失效. 当 $N>1$ 时, $\Delta C_{2n} \neq 0$, 此时故障失效对密文差分值无影响, 该故障失效为隐性故障失效.

2.3 现有攻击方法的不足 陈浩等^[11] 利用二元组 $G = (f_d, \alpha)$ 来表示一个故障位置, 其中 $f_d = 32 - r + 1$ 表示故障深度, r 为故障注入轮, α 表示注入故障的中间状态变量的索引值. 根据 f_d 一定时, α 值不同, 密码产生的错误密文差分 $\Delta C_i \neq 0$ 个数和密文索引值 i 不同, 陈浩等^[11] 构建了一个故障位置区分器, 对故障深度不超过 7 的故障位置进行精准判定. 该故障位置区分器通过定义集合:

$$\Phi(f_d, \alpha) = \{i \mid \Delta C_i \neq 0, 0 \leq i, \alpha \leq 7\}$$

来表征密文差分 $\Delta C_i \neq 0$ 的索引集,进而对故障位置进行判定.例如,图 5 所示为攻击者向密码第 26 加密轮随机注入一个单字节故障时的故障位置和 $\Phi(f_d, \alpha)$ 关系图,由图 5 可知,当在故障传播过程中没有显性故障发生时,故障位置 (f_d, α) 和 $\Phi(f_d, \alpha)$ 一一对应,此时根据 $\Phi(f_d, \alpha)$ 能够非常准确地判定故障位置.

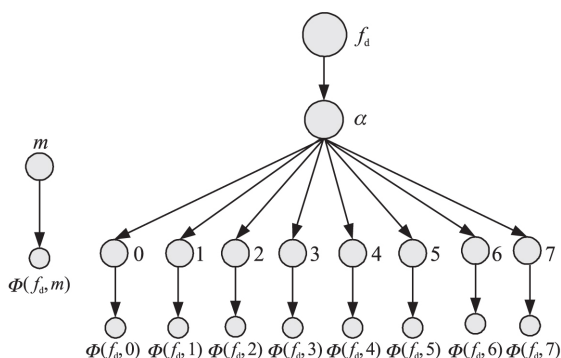


图 5 故障区分器示意图^[11]

Fig.5 Structure of the fault distinguisher^[11]

然而,一旦在故障传播过程中发生显性故障失效时,该故障位置区分器将不能判定出正确的故障位置,进而导致攻击失败.例如,若图 1 中 ① 处发生显性故障失效, $\Delta C_4 = 0$, $\Phi(7, 4) = \{1, 2, 3, 5, 6, 7\}$, 则根据陈浩等^[11]构建的故障位置区分器不能判定出正确的故障位置,攻击失败.此外,陈浩等^[11]还根据具体的故障传播路径构建故障信息等效代数方程组,自动化程度有待提高.

3 HIGHT 容错代数故障攻击

3.1 攻击基本思想 针对已有攻击方法的不足,本文拟从容错性能和自动化程度两个方向对现有方法进行改进.

(1) 容错性能改进: 为了提高攻击的正确率,应尽可能地减少故障失效对攻击的影响.根据之前的分析,在故障传播中故障失效分为隐性故障失效和显示故障失效.如图 1 所示,攻击者向密码加密第 26 轮注入随机单字节故障,中间状态 $X_{30,2}$ 处发生隐性故障失效,由图可知该故障失效对正误密文差分值并未造成影响,攻

击者仍能正确判定故障位置,因此在实际攻击中主要考虑建立新的故障位置区分器来消除或纠正显性故障失效对攻击造成的影响,隐性故障失效则可以忽略不计.

(2) 自动化程度改进: 通常对密码进行代数故障攻击主要分为在线和离线两个阶段^[12],其中在线阶段攻主要完成向密码芯片注入实际随机故障产生错误密文,离线阶段则对注入故障之后芯片产生的数据进行收集,并根据收集到的数据完成故障位置判断、密码整体代数方程组构建、故障信息等效代数方程组构建以及代数方程组求解,进而恢复密码密钥信息.

一个好的代数故障攻击方法自动化程度必须足够高,故障位置的判断、密码整体代数方程组构建和故障信息等效代数方程组构建应能在离线阶段自动完成.基于此,本节将提出高效的故障位置推断和故障信息等效代数方程组构建方法以实现故障位置和故障信息等效代数方程组的自动判定和构建.

3.2 故障位置区分器构建 本小节将以对 26 轮攻击为例,给出新的故障位置区分器的构造方法.分析易知,当攻击者向 $X_{26}, \alpha (\alpha = 2m, 0 \leq m \leq 3)$ 注入故障时,故障传播路径如图 1 所示,可能的显性故障失效点有 4 个: $\{(\textcircled{1}, \textcircled{2}), (\textcircled{3}, \textcircled{4})\}$. 但由于 ① 和 ②, ③ 和 ④ 均不同时发生,因此总的显性故障失效次数为: $C_2^1 + C_2^1 + C_2^1 \times C_2^1 = 8$, 且每个故障失效对应不同的 $\Phi(f_d, \alpha)$, 所以该故障位置对应 8 个不同的 $\Phi(f_d, \alpha)$, $\Phi(f_d, \alpha)$ 具体值可参见附录 1 中的表 3(下同). 同理,当攻击者向 $X_{27}, \alpha (\alpha = 2m + 1, 0 \leq m \leq 3)$ 注入故障时,总的故障失效次数为: $1 + C_2^1 + C_2^1 = 5$, 且每个故障失效对应不同的 $\Phi(f_d, \alpha)$, 所以该故障位置对应 5 个不同的 $\Phi(f_d, \alpha)$. 因此,当攻击者对 26 轮 ($f_d = 7$) 进行攻击时,可构造如图 6 所示的故障位置区分器,完整区分器 ($4 \leq f_d \leq 7$) 可参见附录 1.

在实际攻击中,攻击者输入 $\Phi(f_d, \alpha)$, 故障位置区分器根据 $\Phi(f_d, \alpha)$ 快速自动地判定故障位置 (f_d, α) , 并且标记出发生显性故障失效的中间状态变量.例如,当正误密文差分值为

(00,50,EE,09,00,EA,B7,21)时, $\Phi(f_d, \alpha) = \{1, 2, 3, 5, 6, 7\}$, 故障位置区分器根据输入的 $\Phi(f_d, \alpha)$ 判定故障位置为 (7, 1), 并标记出中间

状态变量 $X_{31,4}$ 在故障传播过程中发生显性故障失效, 其过程如图 6 中虚线部分所示, 整个判定和标记过程是自动完成的。

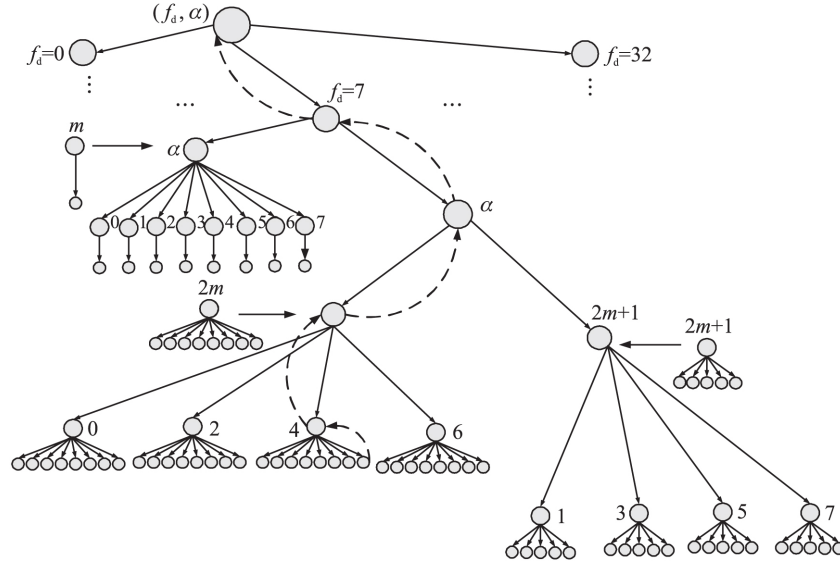


图 6 新构建的故障区分器示意图 ($f_d = 7$)

Fig.6 Structure of the proposed fault distinguisher ($f_d = 7$)

3.3 故障信息等效代数方程组自动化构建

为了叙述方便, 使用下列表达式:

$$A \xrightarrow{(*)} B$$

表示由 A 通过关系式 $(*)$ 推导出 B . 定义集合:

$$\Theta(f_d, \alpha)_n =$$

$$(X_{n,i} \mid \Delta X_{n,i} = 0, 0 \leq i, \alpha \leq 7, r \leq n \leq 32)$$

表示故障传播过程中第 n 轮加密未受影响的中间状态集合, r 为故障注入轮, 且设 $U = \{0, 1, 2, 3, 4, 5, 6, 7\}$, 则显然有:

$$\Omega(f_d, \alpha)_n \cup \Theta(f_d, \alpha)_n = U \quad (26)$$

由密码第 32 轮函数可知:

$$X_{31,m} \xrightarrow{(15) \sim (23)} \begin{cases} X_{32,m}, X_{32,m+1}, & m = 2k, 0 \leq k \leq 3 \\ X_{32,m}, & m = 2k+1, 0 \leq k \leq 3 \end{cases} \quad (27)$$

由密码轮函数可知:

$$X_{i,m} \xrightarrow{(7) \sim (14)} \begin{cases} X_{i+1,m+1}, X_{i+1,(m+2)\%8}, & m = 2k, 0 \leq k \leq 3 \\ X_{i+1,m+1}, & m = 2k+1, 0 \leq k \leq 3 \end{cases} \quad (28)$$

基于式 (26) ~ (28) 知, 当故障注入位置 $X_{r,i}$ 已知时, 集合 $\Theta(f_d, \alpha)_i$ 可以按照如下具体步骤推导出来:

(1) 根据正误密文对 (C, C') 计算 $\Phi(f_d, \alpha)$;

(2) 由 $\Phi(f_d, \alpha)$ 查询故障位置区分器, 得到故障位置 (f_d, α) , 如果故障在传播过程中有中间状态变量发生显性故障失效, 则将其标记并添加至集合 $\Pi(f_d, \alpha)$ 中;

(3) 根据故障位置 (f_d, α) , 利用等式 (18) ~ (20) 迭代计算 $\Theta(f_d, \alpha)_i$, i 的初始值为 $32 - f_d + 1$, 且每次迭代之后 i 自增 1;

(4) 得到 $\Theta(f_d, \alpha)_i$ 后, 判断 i 值是否等于 32, 如果相等则运算结束, 否则跳转到步骤 (3) 继续进行迭代操作。

在推导出 $\Theta(f_d, \alpha)_i$ 后, 为了充分利用故障信息, 实现故障信息的最大化利用和故障信息等效代数方程组的自动化构建, 分别使用 $X_i = X_{i,7} \parallel \dots \parallel X_{i,0}$ 和 $X'_i = X'_{i,7} \parallel \dots \parallel X'_{i,0}$ 表示故障注入前后的中间状态变量, $r \leq i \leq 32$, 则 $\forall X_{i,\varphi} \in \Theta(f_d, \alpha)_i \cup \Pi(f_d, \alpha)$:

$$X_{i,\varphi} \oplus X'_{i,\varphi} = 0 \quad (29)$$

故障信息等效代数方程组自动化构建过程的伪代码如算法 1 所示. 算法 1 输入为故障位置 (f_d, α) , $\Pi(f_d, \alpha)$ 和 $\Theta(f_d, \alpha)_i$, 输出为故障信息等效代数方程组.

算法 1 故障信息等效代数方程组构建

输入: 故障位置 (f_d, α) , $\Pi(f_d, \alpha)$, $\Theta(f_d, \alpha)_r$

输出: 故障信息等效代数方程组

while $X_{i,j} \in \Pi(f_d, \alpha)$ do

$X'_{i,j} \oplus X_{i,j} = 0$

end while

for $i=r; i < 32; i++$

$\Theta(f_d, \alpha)_r = U - \Phi(f_d, \alpha)_r$

while $X_{r,\varphi} \in \Theta(f_d, \alpha)_r$ do

$X'_{r,\varphi} \oplus X_{r,\varphi} = 0$

end while

end for

for $i=r; i < 31; i++$

$X'_{i+1,0} \oplus X'_{i,7} \oplus (F_0(X'_{i,6}) \boxplus SK_{4i+3}) = 0,$

$X'_{i+1,1} \oplus X'_{i,0} = 0$

$X'_{i+1,2} \oplus (X'_{i,1} \boxplus (F_1(X'_{i,0}) \oplus SK_{4i})) = 0,$

$X'_{i+1,3} \oplus X'_{i,2} = 0$

$X'_{i+1,4} \oplus X'_{i,3} \oplus (F_0(X'_{i,2}) \boxplus SK_{4i+1}) = 0,$

$X'_{i+1,5} \oplus X'_{i,4} = 0$

$X'_{i+1,6} \oplus (X'_{i,5} \boxplus (F_1(X'_{i,4}) \oplus SK_{4i+2})) = 0,$

$X'_{i+1,7} \oplus X'_{i,6} = 0$

end for

for $r=31; r < 32; r++$

$X'_{r+1,0} \oplus X'_{r,0} = 0,$

$X'_{r+1,1} \oplus (X'_{r,1} \boxplus (F_1(X'_{r,0}) \oplus SK_{4r})) = 0$

$X'_{r+1,2} \oplus X'_{r,2} = 0,$

$X'_{r+1,3} \oplus X'_{r,3} \oplus (F_0(X'_{r,2}) \boxplus SK_{4r+1}) = 0$

$X'_{r+1,4} \oplus X'_{r,4} = 0,$

$X'_{r+1,5} \oplus (X'_{r,5} \boxplus (F_1(X'_{r,4}) \boxplus SK_{4r+2})) = 0$

$X'_{r+1,6} \oplus X'_{r,6} = 0,$

$X'_{r+1,7} \oplus X'_{r,7} \oplus (F_0(X'_{r,6}) \boxplus SK_{4r+3}) = 0$

end for

$C'_0 \oplus (X'_{32,0} \boxplus WK_4) = 0, C'_1 \oplus X'_{32,1} = 0,$

$C'_2 \oplus X'_{32,2} \oplus WK_5 = 0, C'_3 \oplus X'_{32,3} = 0,$

$C'_4 \oplus (X'_{32,4} \boxplus WK_6) = 0, C'_5 \oplus X'_{32,5} = 0,$

$C'_6 \oplus X'_{32,6} \oplus WK_7 = 0, C'_7 \oplus X'_{32,7} = 0,$

$C' = C'_7 \parallel C'_6 \parallel C'_5 \parallel C'_4 \parallel C'_3 \parallel C'_2 \parallel C'_1 \parallel C'_0$

3.4 攻击复杂度分析 由于本文提出的改进代数故障攻击的复杂度主要由故障注入次数构成, 因此本节主要从理论角度对故障注入次数 N 进行评估.

本质上, 故障分析方法主要利用密码算法正误密文输出差分同故障传播过程中涉及的相关密钥关系进行密钥破解^[13]. 因此, 当且仅当注入的故障在传播过程中涉及全部的主密钥信息 MK 时, 攻击者才能恢复所有主密钥信息 MK .

由陈浩等^[11]的研究结果可知, 当故障注入轮分别为 26 轮和 27 轮时, 恢复全部密钥信息 MK 故障所需的故障注入总数分别为 $N=11$ 和 $N=23$.

3.5 攻击成功率分析 由 3.2 节分析过程可知, 本文提出的新的故障位置区分器具有较强的容错能力, 即使在故障传播过程中发生显性故障失效, 仍能正确判定故障位置, 因此攻击的成功率为 100%. 作为对比, 下面重点从理论角度分析陈浩等^[11]的代数故障攻击方法的成功率.

由 2.2 节故障失效定义可知, 当 $\varphi=0$ 时, $\Delta X_{r+2, 2m+3}$, 故障值 Δ_2 和 Δ_4 相互抵消, 产生的概率为:

$$Pr(\varphi=0) = Pr((\Delta_2 \oplus \Delta_4) = 0) = \frac{1}{255} (\approx 0.0039)$$

当发生显性故障失效时, 会使正误密文差分值发生变化, 从而引起故障位置判定出现错误, 最终导致攻击失败. 假设攻击者向密码 r 轮中间变量注入单字节随机故障, 故障注入位置为 X_r, α , 攻击设定的故障样本为 N , 其中发生在奇数索引的中间变量 ($\alpha=2m+1, 0 \leq m \leq 3$) 和偶数索引中间变量 ($\alpha=2m, 0 \leq m \leq 3$) 的故障在传播过程中发生显性故障失效的次数分别为 s 和 w , 则注入一个故障时, 攻击的成功率为:

$$\begin{aligned} Pr[n=1] &= Pr[\alpha=2m](1-Pr(\varphi=0))^s + \\ &Pr[\alpha=2m+1](1-Pr(\varphi=0))^w = \\ &\frac{1}{2}(0.9961)^s + \frac{1}{2}(0.9961)^w \quad (30) \end{aligned}$$

因为攻击设定的故障样本为 N , 则攻击的成功率为:

$$Pr[n=N] = (Pr[n=1])^N = \left(\frac{1}{2} (0.9961)^s + \frac{1}{2} (0.9961)^w \right)^N \quad (31)$$

此外, 由 3.2 节可知, 当攻击者对密码第 26 轮加密进行攻击, 且故障注入位置为 $X_{26,2m}$ ($0 \leq m \leq 3$) 时, 可能的显性故障失效点为 4 个, 但是①和②, ③和④均不可能同时发生, 因此显性故障失效次数为 2. 同理, 当故障注入位置为 $X_{26,2m+1}$ 时, 显性故障失效次数同样为 2, 因此根据式 (31) 可得攻击成功率为 $Pr(N=7) = 93.76\%$. 同理, 当故障轮为 27 轮时攻击的成功率如表 1 所示.

表 1 攻击所需故障注入总数和成功率

Table 1 Total number of faults required and success rate of the attack

攻击轮	理论故障注入个数	理论攻击成功率	
		文献[11]	本攻击
26	11	93.76%	100%
27	23	91.40%	100%

4 攻击实验及分析比较

在普通 PC 上 (CPU 为 Intel (R), Core (TM) i7-4790 3.20 GHz, 内存 4 G, Windows 7 32 位操作系统), 使用 C++ 编程语言软件模拟实现随机单字节故障注入过程, 使用 CryptoMinisat 4.4 解析器进行求解 (详细使用方法见文献[11]). 为规范攻击成功率, 当解析器求解方程时间大于 3600 s, 输出多个解或无解, 视为攻击失败. 攻击流程如图 7 所示.

下面以对密码第 26 轮加密攻击为例, 给出 HIGHT 容错代数故障攻击实例:

① 随机选取明文 P 和主密钥 MK , 运行密码算法生成正确密文 C , 使用陈浩等^[11]的方法构建正确加密时密码完整的等效代数方程组;

② 向密码 $r=26$ 轮加密任意位置注入单字节的随机故障, 并设定故障样本数 $N=11$, 得到故障 C' , 利用 3.3 节方法自动构建故障信息

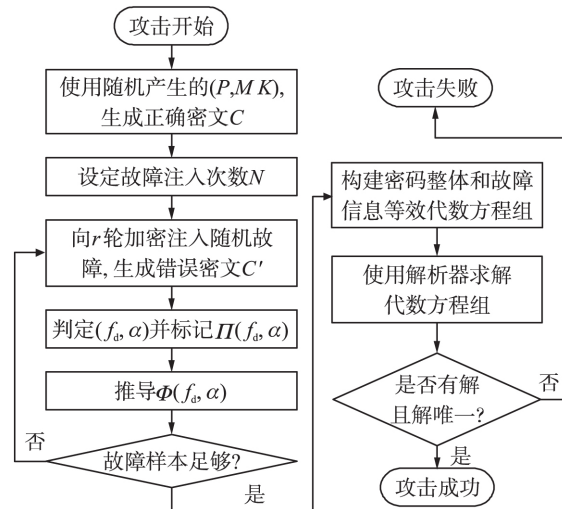


图 7 攻击流程图

Fig.7 Flow-process of the attack

等效代数方程组;

③ 使用 CryptoMinisat 4.4 解析器对构建的代数方程组进行求解得到唯一解:

$MK^* = 2039A71677EFE4D6AA87819CF20068A6$

MK^* 与 MK 相同, 攻击成功, 耗时 5.14 s.

表 2 HIGHT 容错代数攻击实例

Table 2 An instance of the attack on round 26

MK	2039A71677EFE4D6AA87819CF20068A6
P	1D38FEA6731B8790
C	4E5C4BE8F5F96D04
C'_0	745E0CF8F5BCBBEE
C'_1	9EE94BE8E1AB6102
C'_2	4E5C7136DDCA7550
C'_3	4E3B750C42FA850A
C'_4	F20C242A6C73CA1E
C'_5	91B4B3B3C1476D04
C'_6	BB6D035BF574B7E7

在实际攻击中, 分别使用陈浩等^[11]和本文的方法对密码算法第 26 轮加密进行了 100 组攻击仿真实验. 在每个攻击样本中, 分别使用随机产生的主密钥和明文对 (MK, P) 对明文加密生成正确密文, 且均设定故障样本量为 $N=11$,

实验结果如图 8 所示.由图 8 可知,在 100 组攻击样本中,使用陈浩等^[11]的攻击方法的攻击样本中,代数方程组有解且解唯一的样本数为 91 个(9 个失败样本),无解或解个数大于 1 的样本数为 4 个,攻击成功率为 91%,比 3.5 节分析的理论成功率略小,解析器平均求解时间为 2.2738 s;使用本文攻击方法的 100 个攻击样本中代数方程组有解且解唯一的样本数 100 个,攻击实际成功率为 100%,解析器平均求解时间为 2.18 s,攻击结果与 3.5 节理论值相符,验证了方法的正确性.实验结果还表明,使用本攻击方法解析器的平均求解时间为 2.18 s 略小于使用陈浩等^[11]的方法的求解时间 2.2738 s,这说明两种方法都具有较强的有效性,但由于前者在构建故障信息等效代数方程组时,提取了显性故障失效信息,因此求解时间相对来说更小.

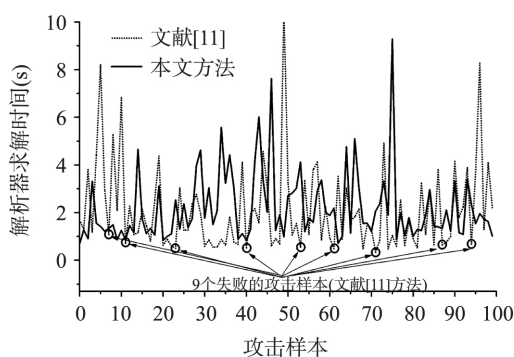


图 8 攻击轮为第 26 轮,故障样本 $N=11$ 攻击结果

Fig.8 Results of the attack on round 26 under $N=11$

此外,针对密码 26 轮加密,还在不同故障样本量 N 下使用本文方法进行了仿真实验,对每个故障样本各进行 100 组仿真实验,实验结果如图 9 所示.由图 9 可知,攻击成功率与故障样本 N 成正比例关系,当设置的故障样本数 N 小于 3 时,攻击成功率为 0%,随着故障样本 N 的逐渐增大,攻击率不断上升,当 N 大于等于 10 时,攻击成功率基本趋于 100%.由图可知,攻击在最好情况下仅需 4 个故障样本即可恢复全部主密钥信息 MK ,解析器平均求解时间为 124.6 s,成功率为 92%,这说明只要注入的故障在传播过程中覆盖了所有的主密钥信

息,攻击就能取得成功,验证了本文方法的高效性.

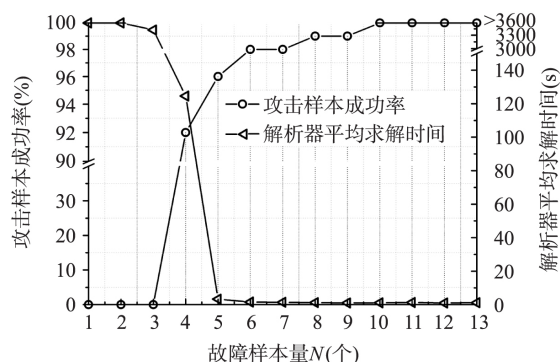


图 9 不同故障样本量下对密码 26 轮的攻击结果

Fig.9 Results of the attack on round 26 under different N

此外,本文还对密码第 27 轮加密进行了 100 次攻击仿真实验,所设定的故障样本分别为 23 次,实验结果表明成功次数为 100 次,攻击成功率为 100%.

对上述实验结果进行综合分析可以发现,攻击的实际成功率主要受显性故障失效和注入的故障在传播过程中是否覆盖所有主密钥信息两个关键因素影响.陈浩等^[11]由于忽略了显性故障失效的存在导致攻击的成功率达不到 100%.此外,在攻击仿真实验中,由于故障注入是通过软件模拟实现的,因此很难实现故障位置完全等概率分布,导致在实验过程中出现当所设定故障样本数小于理论注入次数时攻击仍能取得 100% 成功的现象.为了减小该因素对攻击成功率的影响,使用 6 台 PC 机同时运行大约一周时间,将对密码 26 轮的攻击样本数由 100 组增加到 10000 组,实验结果表明当所设定的故障样本数 $N=10,11,12$ 时,其所对应的攻击成功率分别为 99.96%, 100%, 100%.但由于本文所设计的仿真实验主要用来验证提出的新攻击方法能够从显性故障失效和故障信息等效代数方程组自动化构建两个方面提升攻击的成功率和有效性,因此上述第二个因素对该结论的验证影响较小,可忽略不计.

5 结束语

本文对 HIGHT 轻量级分组密码提出一种容错代数故障攻击,与文献[11]相比,提出的新方法通过建立完备性更强的故障位置区分器来消除故障失效对攻击的影响,将攻击的成功率提升至 100%;通过改进故障信息等代数方程组自动化构建,有效提升了攻击方法的有效性.本文提出的攻击方法可为基于 ARX 结构的轻量级对称密码的实现安全性分析提供参考和借鉴.下一步研究的方向主要着眼于提高攻击方法的实用性,探索将攻击扩展至更深的攻击轮.

参考文献

- [1] Bogdanov A, Knudsen L R, Leander G, *et al.* PRESENT: An ultra-lightweight block cipher. In: Paillier P, Verbauwhede I. Cryptographic Hardware and Embedded Systems — CHES 2007. Springer Berlin Heidelberg, 2007: 4727: 450—466.
- [2] Hong D, Sung J, Hong S, *et al.* HIGHT: A new block cipher suitable for low-resource device. In: Goubin L, Matsui M. Cryptographic Hardware and Embedded Systems — CHES 2006. Springer Berlin Heidelberg, 2006: 4249: 46—59.
- [3] Guo J, Peyrin T, Poschmann A, *et al.* The LED block cipher. In: Preneel B, Takagi T. Cryptographic Hardware and Embedded Systems — CHES 2011. Springer Berlin Heidelberg, 2011: 6917: 326—341.
- [4] Shibutani K, Isobe T, Hiwatari H, *et al.* Piccolo: An ultra-lightweight blockcipher. In: Preneel B, Takagi T. Cryptographic Hardware and Embedded Systems — CHES 2011. Springer Berlin Heidelberg, 2011: 6917: 342—357.
- [5] Courtois N T, Ware D, Jackson K M. Fault-algebraic attacks on inner rounds of DES. In: Strategies Telecom and Multimedia, Montreuil, France; UCL, 2010: 22—24.
- [6] Zhang F, Zhao X J, Guo S Z, *et al.* Improved algebraic fault analysis: A case study on piccolo and applications to other lightweight block ciphers. In: The 2nd International Workshop on Constructive Side-Channel Analysis and Secure Design(COSADE 2013). Springer Berlin Heidelberg, 2013, 7864, 2013: 62—79.
- [7] 吴克辉,赵新杰,王 韬等. PRESENT 密码代数故障攻击. 通信学报, 2012, 33(8): 85—92. (Wu K H, Zhao X J, Wang T, *et al.* Algebraic fault attack on PRESENT. Journal on Communications, 2012, 33(8): 85—92.)
- [8] 赵新杰,郭世泽,王 韬等. Piccolo 密码代数故障分析研究. 计算机学报, 2013, 36(4): 882—894. (Zhao X J, Guo S Z, Wang T, *et al.* Research of algebraic fault analysis on piccolo. Chinese Journal of Computers, 2013, 36(4): 882—894.)
- [9] Mohamed M S E, Bulygin S, Behmann J. Using SAT solving to improve differential fault analysis of Trivium. In: Kim T, Adeli H, Robles R J, *et al.* Information Security and Assurance. Springer Berlin Heidelberg, 2011: 62—71.
- [10] 范伟杰,吴文玲,张 蕾. HIGHT 算法的差分故障攻击. 中国科学院研究生院学报, 2012, 29(2): 271—276. (Fan W J, Wu W L, Zhang L. Differential fault analysis on HIGHT. Journal of Graduate University of Chinese Academy of Sciences, 2012, 29(2): 271—276.)
- [11] 陈 浩,王 韬,张 帆等. HIGHT 密码代数故障分析. 上海交通大学学报, 2015, 49(12): 1817—1825, 1832. (Chen H, Wang T, Zhang F, *et al.* Algebraic fault analysis of HIGHT. Journal of Shanghai Jiao Tong University, 2015, 49(12): 1817—1825, 1832.)
- [12] Joye M, Tunstall M. 密码故障分析与防护. 赵新杰,张世泽,张 帆等译. 北京: 科学出版社, 2015, 239—253. (Joye M, Tunstall M. Fault analysis in cryptography. Zhao X J, Zhang S J, Zhang F, *et al.* Beijing: Science Press, 2015, 239—253.)
- [13] 郭世泽,王 韬,赵新杰. 密码旁路分析原理与方法. 北京: 科学出版社, 2014, 169. (Guo S Z, Wang T, Zhao X J. Principles and methodologies of side-channel analysis in cryptography. Beijing: Science Press, 2014, 169.)

附录 1:

表 3 新构建的故障位置区分器 ($4 \leq f_d \leq 7$)

α	$\Phi(f_d, \alpha)$			
	$f_d=7$	$f_d=6$	$f_d=5$	$f_d=4$
0	$\{0,1,2,3,5,6,7\}$	$\{0,1,4,5,6,7\}$	$\{3,4,5,6,7\}$	$\{2,3,4,5\}$
	$\{0,1,2,3,5,6\}, \{0,1,2,3,5,7\}, \{0,2,3,5,6,7\},$	$\{0,1,4,5,6\}, \{0,1,4,5,7\}, \{0,1,4,6,7\},$	$\{4,5,6,7\},$	
	$\{1,2,3,5,6,7\}, \{0,2,3,5,6\}, \{1,2,3,5,6\},$	$\{0,1,4,6\}, \{0,1,4,7\}$	$\{3,5,6,7\}$	
	$\{1,2,3,5,7\}, \{0,2,3,5,7\}$			
1	$\{0,1,2,3,6,7\}$	$\{0,1,5,6,7\}$	$\{4,5,6,7\}$	$\{3,4,5\}$
	$\{0,1,2,3,6\}, \{1,2,3,6,7\}, \{0,2,3,6,7\},$	$\{0,1,5,6\}, \{0,1,5,7\}$	$\{4,6,7\}$	
	$\{1,2,3,7\}, \{0,2,3,7\}$			
2	$\{0,1,2,3,4,5,7\}$	$\{0,1,2,3,6,7\}$	$\{0,1,5,6,7\}$	$\{4,5,6,7\}$
	$\{0,1,3,4,5,7\}, \{0,1,2,4,5,7\}, \{1,2,3,4,5,7\},$	$\{0,1,2,3,6\}, \{1,2,3,6,7\}, \{0,2,3,6,7\},$	$\{0,1,5,6\},$	$\{4,6,7\}$
	$\{0,2,3,4,5,7\}, \{1,2,4,5,7\}, \{0,2,4,5,7\},$	$\{1,2,3,7\}, \{0,2,3,7\}$	$\{0,1,5,7\}$	
	$\{1,3,4,5,7\}, \{0,3,4,5,7\}$			
3	$\{0,1,2,3,4,5\}$	$\{0,1,2,3,7\}$	$\{0,1,6,7\}$	$\{5,6,7\}$
	$\{0,1,2,4,5\}, \{0,1,3,4,5\}, \{0,2,3,4,5\},$	$\{1,2,3,7\}, \{0,2,3,7\}$	$\{0,1,6\}$	
	$\{0,2,4,5\}, \{0,3,4,5\}$			
4	$\{1,2,3,4,5,6,7\}$	$\{0,1,2,3,4,5\}$	$\{0,1,2,3,7\}$	$\{0,1,6,7\}$
	$\{1,2,3,4,6,7\}, \{1,2,3,5,6,7\}, \{1,2,4,5,6,7\},$	$\{0,1,2,4,5\}, \{0,1,3,4,5\}, \{0,2,3,4,5\},$	$\{1,2,3,7\},$	$\{0,1,6\}$
	$\{1,3,4,5,6,7\}, \{1,2,4,6,7\}, \{1,3,4,6,7\},$	$\{0,2,4,5\}, \{0,3,4,5\}$	$\{0,2,3,7\}$	
	$\{1,2,5,6,7\}, \{1,3,5,6,7\}$			
5	$\{2,3,4,5,6,7\}$	$\{1,2,3,4,5\}$	$\{0,1,2,3\}$	$\{0,1,7\}$
	$\{2,3,4,6,7\}, \{2,3,5,6,7\}, \{2,4,5,6,7\},$	$\{1,3,4,5\}, \{1,2,4,5\}$	$\{0,2,3\}$	
	$\{2,4,6,7\}, \{2,5,6,7\}$			
6	$\{0,1,3,4,5,6,7\}$	$\{2,3,4,5,6,7\}$	$\{1,2,3,4,5\}$	$\{0,1,2,3\}$
	$\{0,1,3,4,5,6\}, \{0,1,3,4,5,7\}, \{0,1,3,4,6,7\},$	$\{2,3,4,6,7\}, \{2,3,5,6,7\}, \{2,4,5,6,7\},$	$\{1,3,4,5\},$	$\{0,2,3\}$
	$\{0,1,3,5,6,7\}, \{0,1,3,4,6\}, \{0,1,3,5,6\},$	$\{2,4,6,7\}, \{2,5,6,7\}$	$\{1,2,4,5\}$	
	$\{0,1,3,4,7\}, \{0,1,3,5,7\}$			
7	$\{0,1,4,5,6,7\}$	$\{3,4,5,6,7\}$	$\{2,3,4,5\}$	$\{1,2,3\}$
	$\{0,1,4,5,6\}, \{0,1,4,5,7\}, \{0,1,4,6,7\},$	$\{4,5,6,7\}, \{3,5,6,7\}$		$\{2,4,5\}$
	$\{0,1,4,6\}, \{0,1,4,7\}$			

(责任编辑 杨可盛)